VERIQTA

AWS

Well-Architected Review:

50 | Real Architecture Decisions

Practical Case Studies, Trade-offs, and Production-Ready Decisions Across the AWS Well-Architected Pillars

Security | Reliability | Performance Efficiency | Cost Optimization | Operational Excellence | Sustainability



@veriqta Instagram | @veriqta GitHub | @veriqta LinkedIn | @veriqta Telegram | @veriqta X | @veriqta Discord

VERIQTA

The Architecture Passed Review, but Nobody Can Operate It

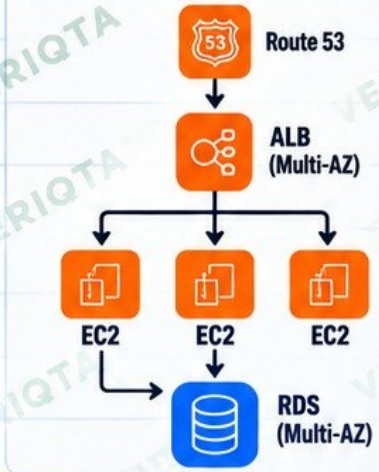
Decision 01

Should a workload be approved when its infrastructure is highly available but its operational ownership is unclear?

Key Points

1. Infrastructure resilience is about keeping the system running. Operational readiness is about who can run, support and recover it.
2. Identify missing service ownership, escalation paths, runbooks, and incident authority.
3. Review operational readiness evidence before accepting the design.
4. A system that nobody can confidently recover is not operationally mature.

Infrastructure Resilience (HIGH AVAILABILITY)



≠

Operational Readiness (WHO RUNS IT?)

- Service ownership (Who owns this service?)
- Escalation path (Who is on call?)
- Runbooks (How to operate and recover?)
- Incident authority (Who can make changes?)
- Operational knowledge (Does the team understand it?)

Senior Engineer Judgment:
A system that nobody can confidently recover is not operationally mature, even if it is highly available.

Decision 02

Should production changes be blocked when rollback is technically possible but untested?

Key Points

1. Distinguish rollback documentation from demonstrated recovery.
2. Examine database migrations, irreversible side effects, and deployment dependencies.
3. Decide when progressive delivery is safer than conventional rollback.
4. Review evidence: previous rollback exercises, deployment history, and recovery duration.

Deployment and Rollback Flow



Key Considerations

- Database migrations (can they be reversed?)
- Irreversible side effects (emails, payments, data changes)
- External dependencies (APIs, third-party systems)
- Rollback time and impact on users

Evidence to Review

- Previous rollback exercises
- Deployment history
- Actual recovery duration
- Success and failure patterns

Progressive Delivery (More Risk Control)

- Canary releases (Small % of traffic)
- Blue/green deployments (Fast switch)
- Feature flags (Gradual exposure)

Conventional Rollback (May Not Always Be Safe)

- May not reverse database changes
- May not undo external effects
- May not work if dependencies have changed

When Infrastructure as Code Becomes an Organizational Risk

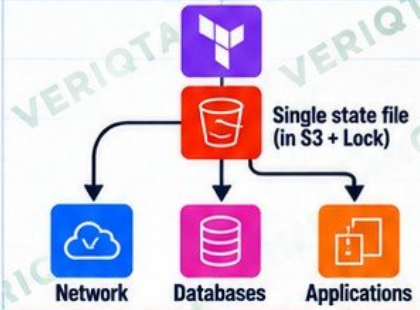
Decision 03

Should Terraform state be centralized across business-critical AWS workloads?

Key Points

1. Compare centralized governance with isolated state and blast-radius boundaries.
2. Evaluate state access, lock contention, credential scope, and accidental resource replacement.
3. Determine where separate state boundaries reduce correlated failures.
4. Architecture trade-off: Operational consistency versus independent recovery.

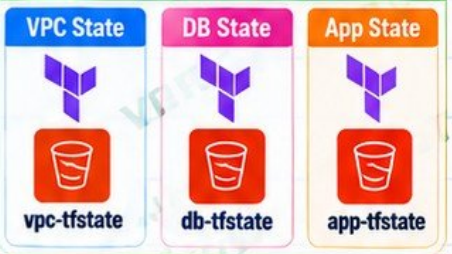
Centralized State (Single Backend)



Risks

- Large blast radius
- Lock contention
- Higher impact of accidental changes
- More complex access control

Isolated State (Separate Backends)



Benefits

- Smaller blast radius
- Independent locking
- Granular access control
- Easier recovery
- Teams can operate independently

Decision 04

Should AWS production infrastructure allow emergency console changes?

Key Points

1. Examine break-glass access, auditability, and incident-time intervention.
2. Understand why strict deployment-only controls may delay urgent recovery.
3. Define temporary exceptions, approval evidence, and reconciliation into infrastructure code.
4. Emergency access needs control, not permanent prohibition or unrestricted privileges.

Emergency Access Flow (Break-Glass)



Why It May Be Needed

- Faster recovery during critical incidents
- Platform or pipeline may be unavailable
- Allows investigation and mitigation
- Prevents prolonged service outage

Controls and Requirements

- Time-limited and audited access
- Approval and justification
- Use dedicated break-glass role
- Mandatory reconciliation into IaC
- Post-incident review

Decision 05

Should every team use the same AWS infrastructure templates?

Key Points

1. Evaluate standardization against workload-specific availability, latency, and compliance needs.
2. Identify when golden paths become restrictive abstractions.
3. Decide which controls are mandatory and which architectural choices remain flexible.
4. Excessive standardization can push teams toward undocumented workarounds.

Benefits of Standard Templates (Golden Path)

- Consistent security controls
- Faster onboarding
- Reduced operational risk
- Easier compliance
- Lower cognitive load
- Standard monitoring and logging

When It Becomes a Problem

- Does not fit workload requirements
- Limits architecture innovation
- Teams create workarounds
- Can increase cost or latency
- May not meet compliance needs
- Reduces team ownership

Recommended Approach

- Define **mandatory** baseline controls (security, tagging, logging, networking)
- Allow flexibility for workload-specific requirements
- Provide a modular template library, not a single rigid template

Your AWS Accounts Are Your Real Security Perimeter

Decision 06

Should production workloads share an AWS account to simplify administration?

Key Points

1. Compare account-level isolation with shared VPC and resource-level separation.
2. Examine IAM blast radius, service quotas, billing visibility, and incident containment.
3. Consider AWS Organizations, service control policies, and account vending.
4. Account boundaries are often stronger isolation mechanisms than naming conventions or tags.

Single Account (Shared)



Risks

- Larger blast radius
- Shared service quotas
- Complex IAM policies
- Harder incident containment
- Mixed billing and cost visibility

Multi-Account (Isolated)



Benefits

- Smaller blast radius
- Clearer security boundaries
- Independent service quotas
- Better cost and billing visibility
- Easier incident isolation

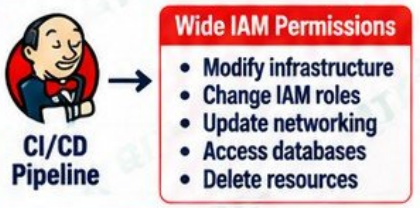
Decision 07

Should CI/CD systems have direct permission to modify production infrastructure?

Key Points

1. Compare broad deployment roles with tightly scoped workload-specific roles.
2. Evaluate short-lived credentials, OIDC federation, and approval boundaries.
3. Investigate how compromised pipelines could alter IAM, networking, or logging.
4. Separate build authority from privileged production deployment authority.

Broad Permissions (High Risk)



Risks

- Pipeline compromise can take over the infrastructure
- Accidental destructive changes
- Difficult to audit and control
- Can alter IAM, networking or logging

Scoped Permissions (Recommended)



Benefits

- Limited blast radius
- Stronger security posture
- Better auditability
- Works with approval workflows
- Reduces risk from compromised pipelines

Senior Engineer Judgment: Separate build authority from privileged production deployment authority. Use scoped, short-lived credentials and enforce approval boundaries for production changes.

VERIQTA

Private Infrastructure Can Still Be an Exfiltration Path

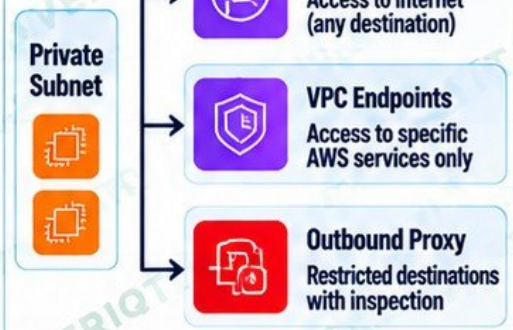
Decision 08

Should every private subnet use a NAT gateway for outbound connectivity?

Key Points

1. Compare NAT gateways, VPC endpoints, and restricted outbound proxies.
2. Evaluate cost, destination control, service dependencies, and failure domains.
3. Investigate how compromised workloads can exfiltrate data through permitted outbound routes.
4. Architecture trade-off: Connectivity simplicity versus controlled egress.

Outbound Connectivity Options



Considerations

- **Cost** (NAT is expensive at scale)
- **Destination control** (Allow or restrict)
- **Service dependencies** (What break if it fails?)
- **Failure domains** (NAT per AZ, proxy HA)

Exfiltration Risk



Compromised workload can exfiltrate data through permitted outbound routes (NAT, proxy, or allowed endpoints).

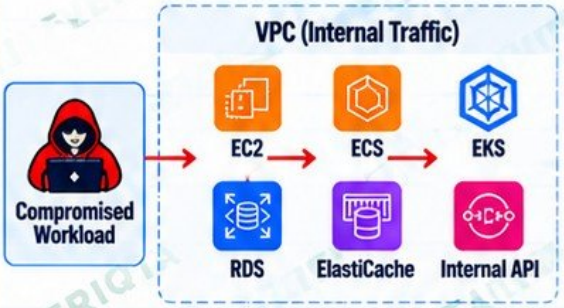
Decision 09

Should internal AWS services trust traffic because it originates inside the VPC?

Key Points

1. Examine east-west traffic, workload identity, and network segmentation.
2. Compare security groups, IAM authorization, and application-layer authentication.
3. Identify risks from compromised workloads and lateral movement.
4. Network location is not sufficient proof of workload identity.

East-West Traffic in a VPC



Controls

- **Security Groups** (Restrict network access)
- **IAM Authorization** (Validate identity and permissions)
- **Application Authentication** (Verify at application layer)

Risks

- Lateral movement
- Access to sensitive data
- Abuse of internal trust
- Harder detection inside VPC

Decision 10

Should application teams control their own encryption keys?

Key Points

1. Compare AWS-managed and customer-managed KMS keys.
2. Evaluate key policies, grants, rotation, cross-account access, and operational ownership.
3. Examine the blast radius of disabled keys or inaccessible encrypted resources.
4. Greater cryptographic control introduces additional availability responsibilities.

KMS Key Options

AWS-Managed Key (AWS owns and manages)

- Simpler operations
- Automatic rotation
- No manual key management
- Limited customization
- Suitable for many workloads

Customer-Managed Key (You own and manage)

- Full control and flexibility
- Custom key policies and grants
- Cross-account access
- Manual rotation and monitoring
- Greater operational responsibility

Risks of Customer-Managed Keys

- Disabled or deleted key
- Lost access or incorrect key policy
- Cross-account permission issues
- Service outage due to key unavailability
- Larger blast radius for encrypted data

Use Cases

- Regulatory or compliance requirements
- Cross-account encryption
- Need for granular access control
- Customer-owned data governance

The Security Control That Caused the Production Outage

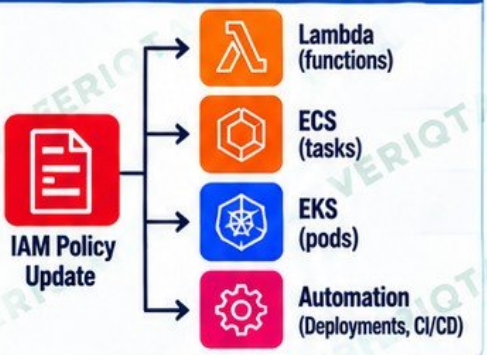
Decision 11

Should IAM permission changes be deployed immediately across every account?

Key Points

1. Investigate policy propagation, dependency discovery, and rollback challenges.
2. Assess how restrictive changes affect Lambda, ECS, EKS, and automated deployments.
3. Design staged policy enforcement and access testing.
4. Security improvements require deployment safety just like application changes.

IAM Policy Change Impact



Risks of Immediate Deployment

- Breaks running services
- Hidden dependencies
- Failed deployments
- Rollback is complex
- Propagation delays across accounts and regions

Recommended Approach

- Discover and test affected resources
- Use staged rollout (dev → staging → prod)
- Validate with access testing and monitoring
- Have a rollback plan and quick recovery process

Operational Evidence to Review

- IAM policy change history
- CloudTrail events and errors
- Service access failures
- Deployment pipeline results
- Time to detect and recover

Decision 12

Should secrets rotate automatically when applications cannot tolerate credential overlap?

Key Points

1. Examine AWS Secrets Manager rotation, connection pooling, and credential caching.
2. Identify failure windows when applications continue using invalid credentials.
3. Compare coordinated rotation with overlapping credential strategies where supported.
4. Review operational evidence: authentication errors, connection resets, rotation events, and application recovery.

Secrets Rotation Flow (AWS Secrets Manager)



Common Causes of Outages



Services Commonly Affected



Rotation Strategies

Without Overlap (May cause outage)

- Old credential invalidated
- Applications must refresh immediately
- Higher risk if connections are long-lived

With Overlap (Where supported)

- Old and new credentials valid at the same time
- Allows gradual refresh
- Reduced risk of connection failures

Operational Evidence

- Authentication errors in logs
- Connection resets and timeouts
- Secrets rotation events and timing
- Application recovery duration
- Impact on dependent services

Multi-AZ Does Not Mean Your Application Is Highly Available

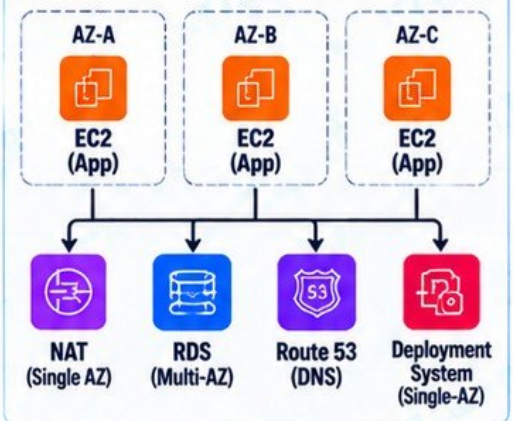
Decision 13

Should a workload be approved as highly available simply because it runs across three Availability Zones?

Key Points

1. Investigate hidden single points of failure in NAT, databases, DNS, and deployment systems.
2. Evaluate whether application dependencies share the same failure domains.
3. Test whether the remaining capacity can handle traffic after an AZ failure.
4. Geographic distribution means little without dependency resilience.

Example: Multi-AZ Architecture with Hidden Single Points of Failure



What to Investigate

- Is NAT a single point of failure?
- Do databases truly fail over?
- Is DNS highly available?
- Are deployment systems multi-AZ?
- Do all dependencies span multiple AZs?

Validation

- Simulate an AZ failure
- Measure remaining capacity
- Verify dependency behaviour
- Confirm traffic can still be served

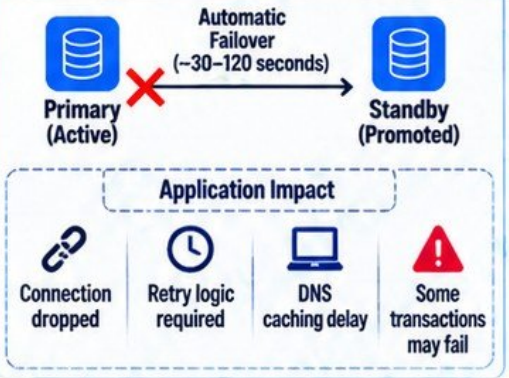
Decision 14

Should production applications fail over automatically when a database becomes unavailable?

Key Points

1. Examine RDS Multi-AZ failover behavior and connection interruption.
2. Evaluate connection retry logic, transaction safety, and DNS caching.
3. Distinguish database availability from application recovery.
4. Review evidence: failover duration, connection errors, and recovery success.

RDS Multi-AZ Failover Flow



Considerations

- Failover duration varies
- Active connections are terminated
- Applications must retry connections
- DNS may cache old endpoint
- Transactions may need to be retried or rolled back

Good Practices

- Implement connection retry with backoff
- Use short DNS TTL where appropriate
- Design for idempotency and transaction safety
- Test failover in non-production environments

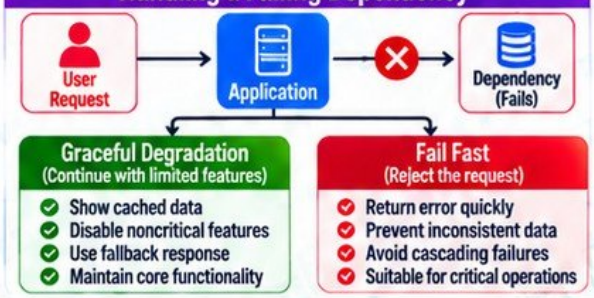
Decision 15

Should applications continue serving traffic when a noncritical dependency fails?

Key Points

1. Compare fail-fast behavior with graceful degradation.
2. Examine feature isolation, fallback responses, and circuit breakers.
3. Identify when fallbacks hide failures or return dangerously stale information.
4. Partial functionality versus strict correctness.

Handling a Failing Dependency



When to Use

Graceful Degradation

- Noncritical features
- Read-heavy or cached data
- Improved user experience
- Maintain core functionality

Fail Fast

- Critical operations (payments, orders)
- Risk of stale or incorrect data
- Compliance or security sensitive
- Maintains data integrity

The Disaster Recovery Plan That Cannot Recover the Business

Decision 16

Should a critical workload use multi-region active-active architecture?

Key Points

1. Compare active-active, active-passive, warm standby, and backup-and-restore.
2. Examine cross-region replication, routing, state consistency, and operational complexity.
3. Assess whether recovery time and recovery point objectives justify the expense.
4. Multi-region complexity can introduce failures that a simpler architecture avoids.

Multi-Region Architecture Options

Active-Active



- Traffic in multiple regions
- Lowest RTO/RPO
- Highest complexity and cost

Active-Passive



- One active, one standby
- Faster failover
- Lower cost than active-active

Warm Standby



- Scaled-down environment
- Moderate RTO/RPO
- Lower cost, some setup time

Backup and Restore



- Data backups only
- Highest RTO/RPO
- Lowest cost, manual recovery

Key Considerations

- Cross-region replication (data, databases, state)
- Routing (Route 53, health checks, failover)
- State consistency and conflict handling
- Operational complexity (monitoring, testing, runbooks)
- Cost vs. business requirements (RTO/RPO)

Decision 17

Should database backups count as sufficient disaster recovery evidence?

Key Points

1. Examine backup integrity, restore permissions, encryption dependencies, and recovery duration.
2. Test recovery into an isolated environment.
3. Identify missing infrastructure, configuration, and external dependencies.
4. Measure actual restoration capability, not backup-job success.

Backup vs. Real Recovery

Backup Job Success

- Backup completed
- No errors reported
- Stored in S3 or backup vault
- Does NOT mean you can recover

Actual Recovery Test

- Restore into isolated environment
- Verify application functionality
- Check permissions and KMS keys
- Validate dependencies (network, IAM, config)
- Measure recovery time (RTO)
- Confirm data integrity (RPO)

Common Missing Dependencies

Compute Infrastructure | Networking (VPC, subnets, DNS) | Configuration (IaC, environment) | Encryption Keys (KMS) | External Services (APIs, third-party)

Validation Checklist

- Can backups be restored?
- Do we have required IAM permissions?
- Are KMS keys accessible?
- Can the application start successfully?
- Are external dependencies available?
- How long does full recovery take?
- Is the data consistent and complete?

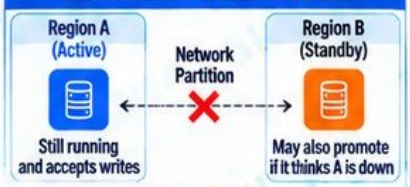
Decision 18

Should failover be automated when the system cannot reliably distinguish an outage from a network partition?

Key Points

1. Investigate health-check ambiguity, stale observations, and split-brain risks.
2. Evaluate authoritative ownership and write fencing.
3. Compare automatic failover with controlled human intervention.
4. Faster recovery versus the risk of conflicting writers.

The Network Partition Problem



Risk: Split Brain

- Both regions accept writes
- Data divergence and inconsistency
- Difficult and complex recovery

Approaches

Automatic Failover

- Faster recovery
- Requires reliable failure detection
- Risk of split brain during network partition

Controlled Human Intervention

- Reduces risk of conflicting writers
- Allows manual verification
- Slower recovery time

Key Considerations

- Health-check accuracy (false positives/negatives)
- Stale observations (delayed metrics)
- Authoritative ownership (which region can write)
- Write fencing (prevent concurrent writes)
- Business impact of failover delay

Your Reliability Mechanisms Are Causing Cascading Failures

Decision 19

Should every microservice retry failed downstream requests?

Key Points

1. Model retry multiplication across service dependencies.
2. Examine timeout budgets, backoff, jitter, and retry limits.
3. Identify how retry storms can overwhelm recovering services.
4. Retry policies must respect end-to-end request budgets.

How Retry Multiplication Happens



1 request → 3x → 9x → 27x → 81x

A single failed request can become 81 requests to the downstream service.

Retry Best Practices

- Use sensible retry limits
- Apply exponential backoff with jitter
- Set appropriate timeouts
- Consider end-to-end request budgets
- Use circuit breakers
- Monitor downstream service health and saturation

Risks

- Retry storms during recovery
- Increased latency and queue depth
- Cascading failures across services
- Exhausted downstream resources

Decision 20

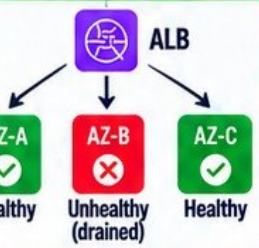
Should application traffic automatically shift away from an unhealthy Availability Zone?

Key Points

1. Examine load balancer health checks, target capacity, and routing behavior.
2. Evaluate the risk of overwhelming healthy zones.
3. Compare evacuation strategies with preserving degraded local capacity.
4. Evidence: target health, request failures, capacity headroom, and latency.

Traffic Shift Scenarios

Automatic AZ Evacuation



- Stops traffic to unhealthy AZ quickly
- Can overload remaining AZs
- Requires sufficient spare capacity
- May increase latency for some users

Preserve Degraded Capacity



- Keeps local capacity available
- Reduces sudden traffic surge
- Users may experience errors
- Suitable for noncritical workloads
- Requires careful health thresholds

What to Evaluate

- Target health and failure thresholds
- Request failure rates
- Capacity headroom in healthy AZs
- Latency impact during shift
- Application behaviour during partial outage

Risks

- Overwhelming healthy zones
- Cascading failures from sudden load
- Increased latency and errors
- Unintended full regional impact

Decision 21

Should critical systems always favor availability over consistency?

Key Points

1. Compare stale reads, duplicate writes, and conflicting state updates.
2. Evaluate consistency requirements for payments, inventory, and identity systems.
3. Identify operations that can degrade safely and those that must reject requests.
4. Define consistency guarantees per business operation.

Consistency vs Availability Trade-off

Favor Availability (Eventual Consistency)

- Continues serving requests
- May return stale data
- Possible duplicate writes
- Used for noncritical features (e.g., product catalog, analytics)

Favor Consistency (Strong Consistency)

- Rejects or delays requests
- Prevents conflicting updates
- Ensures correct state
- Required for critical operations (e.g., payments, inventory, identity)

Examples by Use Case



When Availability Is Acceptable

- Noncritical features
- Read-heavy workloads
- Data can be refreshed later
- User experience is more important than immediate consistency

When Consistency Is Required

- Payments and financial transactions
- Inventory and order management
- Identity and access control
- Operations where incorrect data causes business or legal risk

Senior Decision

Define consistency guarantees per business operation. Not all systems should favor availability over consistency.

The Centralized Network That Became a Shared Outage

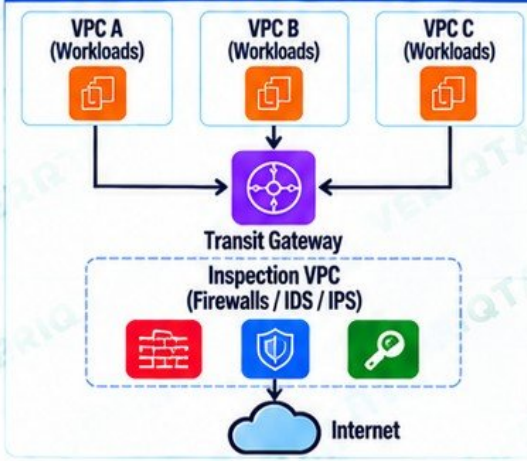
Decision 22

Should every VPC route through a centralized inspection architecture?

Key Points

1. Compare centralized inspection with distributed security controls.
2. Examine AWS Transit Gateway, inspection VPCs, routing symmetry, and appliance capacity.
3. Identify shared failure domains and cross-AZ data-transfer costs.
4. Architecture trade-off: Centralized visibility versus operational dependency concentration.

Centralized Inspection Architecture



Considerations

- Single point of failure risk
- Appliance capacity and scaling
- Routing symmetry requirements
- Cross-AZ data transfer costs
- Operational complexity
- Visibility and consistent controls

Alternatives

- Distributed security controls per VPC
- Selective inspection (only specific traffic)
- NAT gateways with security services
- Service-level controls (e.g., VPC endpoints)

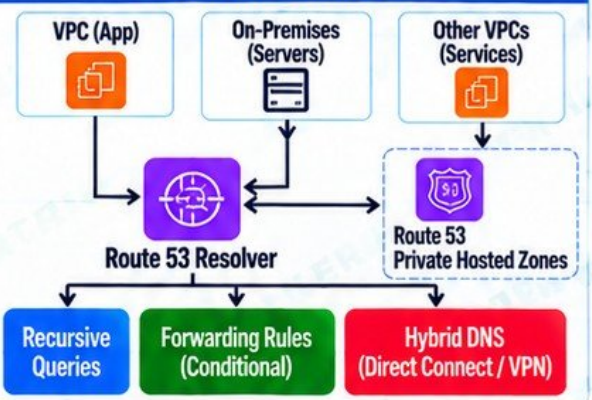
Decision 23

Should AWS workloads rely on one shared DNS architecture?

Key Points

1. Investigate Route 53 Resolver rules, private hosted zones, and hybrid DNS forwarding.
2. Examine forwarding loops, stale records, and unreachable DNS endpoints.
3. Determine how DNS failures propagate across otherwise healthy services.
4. DNS must be included in dependency and disaster recovery reviews.

Shared DNS Architecture



Common DNS Issues

- Forwarding loops
- Stale DNS records
- Unreachable DNS endpoints
- Increased latency
- Failures affecting multiple VPCs

Best Practices

- Use highly available resolvers
- Monitor DNS health and query logs
- Avoid single points of failure
- Test DNS failure scenarios

Decision 24

Should production services use PrivateLink instead of VPC peering?

Key Points

1. Compare PrivateLink, VPC peering, and Transit Gateway.
2. Evaluate connectivity scope, transitive routing, exposure, and service ownership.
3. Assess operational complexity and per-endpoint costs.
4. Choose connectivity based on trust boundaries and required communication patterns.

Connectivity Options Comparison

AWS PrivateLink

- Private, secure connectivity
- No need for transitive routing
- Service owner controls access
- Ideal for shared services (SaaS, internal services)
- Per-endpoint cost

VPC Peering

- Direct VPC-to-VPC connectivity
- No transitive routing
- Simple and low cost
- Requires non-overlapping CIDR blocks
- Less control over service access

Transit Gateway

- Connect many VPCs and on-premises networks
- Supports transitive routing
- Centralized management
- Higher operational complexity
- Higher cost at scale

When to Use PrivateLink

- Cross-account service access
- Strict network isolation required
- Expose services to specific consumers
- Simpler routing and security model

When to Use Peering or Transit Gateway

- Many VPCs need to communicate
- Transitive routing is required
- Lower cost for simple connectivity
- You control both sides of the connection

The Network Is Healthy, but Requests Still Time Out

Decision 25

Should an application increase timeouts when users experience intermittent failures?

Key Points

1. Examine TCP retransmissions, connection reuse, packet loss, and downstream saturation.
2. Distinguish network delay from application queueing.
3. Correlate VPC Flow Logs, load balancer metrics, and application traces.
4. Increasing timeouts without identifying the bottleneck may worsen resource exhaustion.

Where Timeouts Happen



Investigate and Correlate

VPC Flow Logs
- Rejected traffic
- Latency
- Packet loss

Load Balancer Metrics
- Target response time
- HTTP 4XX/5XX
- Connection count
- Unhealthy targets

Application Traces
- Service latency
- Queueing time
- Downstream calls
- Error rates

Common Root Causes

- Downstream service saturation
- Database connection limits
- Network packet loss or retransmissions
- Application queueing or thread exhaustion
- Slow external API dependencies
- Misconfigured timeouts or connection pools

Why Increasing Timeouts Can Be Harmful

- More concurrent requests and resource usage
- Longer queueing and higher latency for all users
- Connection pools become exhausted
- Hides the real bottleneck
- Can trigger cascading failures

Better Approaches

- Identify and fix the actual bottleneck
- Use appropriate timeout budgets per dependency
- Implement connection pooling and reuse
- Use retries with backoff and jitter where safe
- Monitor end-to-end latency and error rates
- Set alerts on downstream saturation and queue depth

Decision 26

Should an Application Load Balancer terminate TLS for every service?

Key Points

1. Compare TLS termination at the load balancer with encryption to backend targets.
2. Evaluate certificate ownership, workload authentication, and compliance requirements.
3. Examine the security implications of plaintext backend connections.
4. Architecture trade-off: Operational simplicity versus end-to-end encryption requirements.

TLS Termination Options

Terminate TLS at ALB (Decrypt at Load Balancer)



- Simpler certificate management
- Offloads TLS from targets
- Common for many web applications
- Targets receive plaintext traffic
- Requires trusted network between ALB and targets

End-to-End Encryption (TLS to Targets)



- Encrypts traffic to backend targets
- Stronger security and compliance
- Target authentication (mTLS) possible
- Higher operational complexity
- Requires certificate management on targets

Considerations

- Certificate ownership and rotation
- Workload authentication (mTLS)
- Compliance requirements (e.g., PCI DSS, HIPAA)
- Security of backend traffic
- Network architecture and trust boundaries
- Operational complexity and cost

Common Use Cases

ALB Termination (Common)
- Public web applications
- Multiple services behind one ALB
- Centralized certificate management

End-to-End Encryption (When Required)
- Sensitive or regulated workloads
- Service-to-service authentication (mTLS)
- Zero trust or strict security environments

EKS Is Not Automatically the Right Platform

Decision 27

Should every containerized application run on Amazon EKS?

Key Points

1. Compare EKS, ECS, Fargate, and simpler managed compute options.
2. Evaluate control-plane responsibilities, portability, operational skills, and costs.
3. Determine whether Kubernetes flexibility justifies the complexity.

Platform Comparison

EKS	ECS	Fargate	Other Compute
Full Kubernetes	AWS managed	Serverless containers	EC2 / Lambda / App Runner
High flexibility	Simpler to operate	No node management	Simpler and lower cost
Higher operational overhead	Lower cost	Pay per use	Good for specific workloads
Higher cost	Less flexibility than EKS	Limited control	

Senior Decision

Choose the least complex platform that satisfies real workload requirements.

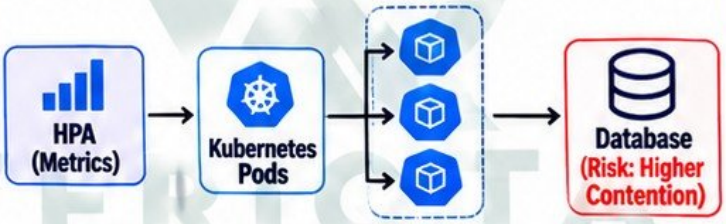
Decision 28

Should Kubernetes workloads use aggressive horizontal autoscaling?

Key Points

1. Examine HPA metrics, startup delays, and scheduling constraints.
2. Identify why adding Pods may increase database contention.
3. Compare reactive scaling with predictive capacity planning.

Autoscaling Behavior



Scale Up (More Pods) ≠ Always Better Performance

Observability

- Scaling lag (time to add new Pods)
- Resource saturation (CPU, memory, DB)
- Queue depth (request backlog)
- Request latency (p95 / p99)

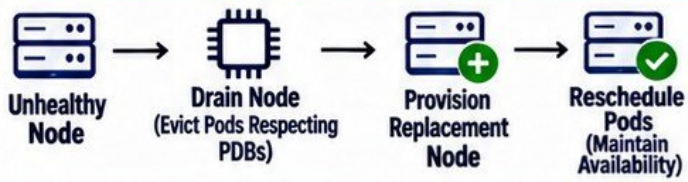
Decision 29

Should EKS nodes be replaced automatically whenever they become unhealthy?

Key Points

1. Examine PodDisruptionBudgets (PDBs), drain behavior, and termination grace periods.
2. Evaluate replacement capacity and disruption overlap.
3. Identify how simultaneous remediation can cause a larger outage.

Node Replacement Flow



Risk to Manage

- Stateful workloads with data dependencies
- Insufficient replacement capacity
- Multiple nodes remediated at the same time
- Inadequate termination grace periods

Reliability Decision

Coordinate node recovery with workload availability requirements.

Serverless Removes Servers, Not Architecture Problems

Decision 30

Should a latency-sensitive application move from containers to AWS Lambda?

Key Points

1. Examine cold starts, concurrency limits, and execution duration.
2. Evaluate cost for bursty vs. sustained workloads.
3. Consider downstream database connection exhaustion.
4. Weigh operational simplicity against execution constraints.

Containers vs. AWS Lambda

Containers (ECS/EKS)

- Long-running workloads
- Predictable performance
- Full control over runtime
- Higher operational overhead

VS

AWS Lambda

- No server management
- Auto scaling (per request)
- Pay per execution
- Cold starts and limits (≤ 15 minutes)

Trade-Off

Operational simplicity vs. execution constraints (latency, duration, connections, cost).

Decision 31

Should asynchronous workflows use SQS instead of synchronous service calls?

Key Points

1. Compare latency, delivery guarantees, and failure isolation.
2. Examine duplicate messages, visibility timeouts, and DLQs.
3. Determine where idempotency is essential.

Synchronous vs. Asynchronous (SQS)

Synchronous (Service Call)

Client → Service A → Service B

- Tighter coupling
- Failures propagate
- Higher latency risk

Asynchronous (SQS)

Client → Amazon SQS → Worker (Service B)

- Decoupled services
- Buffers traffic
- Improved resilience
- Handles spikes

Use SQS when you need decoupling, resilience, and queue-based processing.

Decision 32

Should every business workflow use AWS Step Functions?

Key Points

1. Compare explicit orchestration with event-driven choreography.
2. Evaluate state transitions, failure handling, service integration, and cost.
3. Identify risks from long-running workflows and partial external side effects.

Workflow Options

AWS Step Functions (Orchestration)

- Centralized workflow control
- Built-in retries and error handling
- Service integrations (AWS and HTTP)
- Clear state and execution history

Event-Driven Choreography

- Looser coupling between services
- Lower cost and simpler design
- Independent service ownership
- No central workflow dependency

Senior Decision

Use orchestration when coordination, visibility, and reliable failure handling justify central control.

The Database Is Not Slow. The Architecture Is Overloading It

Decision 33 — Should application teams increase RDS instance size when database latency rises?

Key Points

1. Investigate lock contention, inefficient queries, and connection pressure.
2. Separate compute bottlenecks from poor query behavior.
3. Evaluate read replicas, caching, and connection pooling.

Root Cause vs. Solution

Amazon RDS →

Common Real Causes of High Latency

- ✗ Lock contention
- ✗ Inefficient queries (e.g., full table scans)
- ✗ Too many connections
- ✗ I/O saturation (throughput or IOPS)

→

Better Options Before Scaling

- ✓ Optimize queries and indexes
- ✓ Use read replicas (for read traffic)
- ✓ Add/scale caching (e.g., ElastiCache)
- ✓ Implement connection pooling

Senior Judgment

Scaling resources does not fix inefficient access patterns.

Decision 34 — Should DynamoDB replace relational databases for high-throughput workloads?

Key Points

1. Compare access-pattern-driven design with relational flexibility.
2. Examine hot partitions, consistency requirements, and transaction complexity.
3. Evaluate how changing query patterns affect future maintainability.

RDS vs. DynamoDB

Relational (RDS / Aurora)

- ✓ Flexible queries (SQL)
- ✓ Complex transactions
- ✓ Strong consistency
- ✓ Better for multi-entity relationships

VS

Amazon DynamoDB

- ✓ High, predictable throughput
- ✓ Access-pattern-driven design
- ✓ Horizontal scale (milliseconds)
- ✓ Lower operational overhead

Architecture Trade-Off

Predictable access performance vs. data-model flexibility.

Decision 35 — Should applications use ElastiCache to reduce database load?

Key Points

1. Examine cache invalidation, stale data, hot keys, and eviction.
2. Identify cache stampedes during recovery.
3. Evaluate read-through, write-through, and cache-aside strategies.

Caching Architecture and Strategies

Application ⟷ Amazon ElastiCache (Redis / Memcached) ⟷ Database (RDS / Aurora)

Caching Strategies

- ✓ Cache-aside (most common)
- ✓ Read-through (on cache miss)
- ✓ Write-through (keep cache updated)

Risks to Manage

- ✗ Stale data (consistency)
- ✗ Hot keys (skewed traffic)
- ✗ Cache stampede (thundering herd)
- ✗ Eviction (loss of important data)

Senior Decision

Treat caching as a consistency and failure-management decision, not merely a performance improvement.

The Fastest Architecture Is Not Always the Most Efficient

Decision 36 — Should applications run on larger EC2 instances to improve latency?

Key Points

1. Examine CPU scheduling, memory pressure, network bandwidth, and workload contention.
2. Compare vertical scaling with horizontal scaling.
3. Identify whether performance gains justify reduced placement flexibility.
4. Use evidence: tail latency, utilization, throttling, and saturation.

Vertical vs. Horizontal Scaling

EC2 (Larger Instance)

- ✓ More CPU, memory, and network capacity
- ✓ Quick to implement
- ✓ Can reduce latency for single-node workloads
- ✗ Higher cost
- ✗ Scaling limits (instance size ceiling)
- ✗ Reduced placement flexibility

VS

EC2 (Horizontal Scaling)

- ✓ Scales out with demand
- ✓ Higher availability and fault isolation
- ✓ Better placement and resilience
- ✗ More operational complexity
- ✗ Requires a load balancer and stateless design

Evidence to Review

- Tail latency (p95 / p99)
- Resource utilization (CPU, memory)
- Throttling (e.g., CPU credit, I/O)
- Saturation (CPU, memory, network)

Decision Insight: Use larger instances only when the workload is resource-constrained and the performance gains are measurable and cost-justified.

Decision 37 — Should global applications use CloudFront for all content delivery?

Key Points

1. Compare edge caching, dynamic origin requests, and regional delivery.
2. Evaluate cache-key design, invalidation, data sensitivity, and origin protection.
3. Examine when caching improves latency but creates correctness risks.

Using Amazon CloudFront

Global Users ⟷ Amazon CloudFront (Edge Caching) ⟷ Origin (ALB / EC2 / S3 / API)

Benefits

- ✓ Lower latency (global edge)
- ✓ Reduced origin load and cost
- ✓ Better availability and DDoS protection (AWS Shield)
- ✓ Supports static and dynamic content (with control)

Risks & Considerations

- ✗ Stale or incorrect data (if caching is misconfigured)
- ✗ Cache-key design (headers, cookies, query strings)
- ✗ Invalidation delays and cost
- ✗ Not suitable for sensitive or personalized data

Senior Judgment

Use CloudFront where it delivers clear performance and cost benefits without compromising data freshness, security, or correctness.

Capacity Planning Beyond CPU Utilization

Decision 38

Should scaling policies use CPU utilization as their primary signal?

Key Points

1. Compare CPU-based scaling with request concurrency, queue age, and custom business metrics.
2. Identify workloads limited by network, memory, or downstream services.
3. Examine leading indicators of saturation.

Scaling Signal Comparison

CPU-Based Scaling

CPU Utilization

- ✓ Simple and widely used
- ✓ Works for CPU-bound tasks
- ✗ Misses non-CPU bottlenecks (network, memory, DB)
- ✗ May scale too late for real demand

VS

Alternative Signals

- ✓ Request concurrency (in-flight requests)
- ✓ Queue age / backlog (e.g., SQS, Kafka)
- ✓ Custom business metrics (e.g., orders/min, users)
- ✓ Resource-specific signals (memory, network, I/O)

Senior Decision

Scale against the resource that constrains useful throughput, not just CPU utilization.

Leading Indicators of Saturation

- Rising latency (p95/p99)
- Increasing queue depth / age
- Higher memory pressure
- Network throughput near limit
- Slower downstream (e.g., database)
- Error rate or timeouts increasing

Decision 39

Should applications reserve substantial unused capacity for peak demand?

Key Points

1. Compare reserved headroom, scheduled scaling, and reactive autoscaling.
2. Evaluate unpredictable traffic spikes and scaling lead time (e.g., instance provisioning, warm-up).
3. Calculate the operational consequences of insufficient headroom (e.g., latency, errors, SLA breach).

Capacity Planning Options

Reserved Headroom

- ✓ Always available capacity
- ✓ Fastest response to traffic spikes
- ✗ Higher steady-state cost

Scheduled Scaling

- ✓ Pre-scale for known events (e.g., sales, campaigns)
- ✓ Balances cost and readiness
- ✗ Requires accurate traffic forecasting

Reactive Autoscaling

- ✓ Scales based on real-time demand (e.g., CPU, queue, requests)
- ✓ Cost efficient for variable traffic
- ✗ Scaling lag can cause temporary degradation

Trade-Off

Higher steady-state cost versus faster response to demand.

Planning Formula

Required Capacity = Baseline Demand + Peak Uplift + Safety Headroom
(e.g., 20% – 40% based on SLA, traffic volatility, and scaling lead time)

The Cheapest AWS Design Can Become the Most Expensive Incident

Decision 40

Should production workloads use Spot Instances to reduce compute costs?

Key Points

1. Examine Spot interruption behavior and reclaim timelines.
2. Determine which workloads can tolerate termination.
3. Evaluate checkpointing, state recovery, and capacity variability.
4. Compare Spot, On-Demand, and Savings Plans strategies.

EC2 Purchasing Options Comparison

Spot Instances

- ✓ Lowest cost (up to 70–90% off)
- ✓ Best for stateless and fault-tolerant workloads
- ✗ Can be interrupted (with short notice)
- ✗ Not suitable for stateful or critical jobs

On-Demand Instances

- ✓ Highest flexibility (no commitment)
- ✓ Best for predictable or critical workloads
- ✗ Highest cost

Savings Plans (1–3 Years)

- ✓ Lower cost (≈ 30–72% off)
- ✓ Commit to consistent usage (EC2, Fargate, etc.)
- ✓ Good for steady or baseline workloads

Senior Judgment

Use Spot Instances only when the workload can be interrupted, recovered, and rescheduled without business impact.

Spot Workload Pattern (Example)

Batch Job / Worker → Checkpoint (S3 / EFS) → Interruption (Spot Reclaimed) → Restart (from Checkpoint)

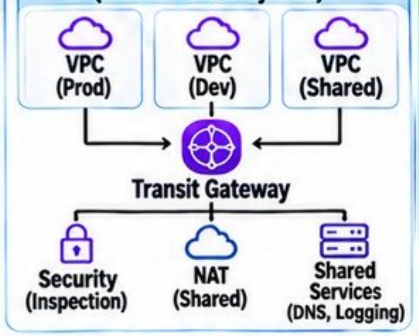
Decision 41

Should a company centralize all AWS networking to reduce infrastructure duplication?

Key Points

1. Examine Transit Gateway charges, NAT processing, and cross-AZ data transfer.
2. Evaluate shared service dependencies (e.g., security, DNS, inspection).
3. Compare centralized networking with distributed VPC designs.
4. Analyze traffic patterns and total cost impact (not just resource counts).

Networking Design Comparison



Centralized (Pros & Cons)

- ✓ Simpler security and governance
- ✓ Centralized logging and inspection
- ✗ Higher Transit Gateway and data transfer costs
- ✗ Potential single point of dependency

Distributed (Pros & Cons)

- ✓ Lower cross-VPC data transfer cost
- ✓ Greater isolation and flexibility
- ✗ More networking components to manage
- ✗ Potential duplication (NAT, endpoints, policies)

Senior Decision

Choose the networking model that minimizes total traffic cost while meeting security, operational, and isolation requirements.

Key Takeaway

A design that looks cheaper on paper (e.g., Spot or centralized networking) can become more expensive when you factor in recovery time, disruption impact, and traffic costs.

Cloud Cost Optimization Without Damaging Reliability

Decision 42

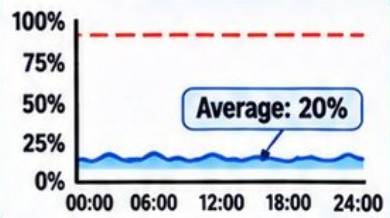
Should production databases be downsized when average utilization is low?

Key Points

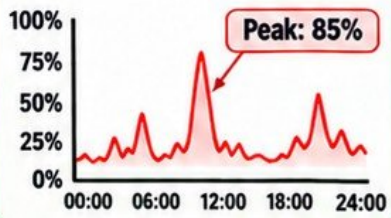
1. Examine peak utilization, memory working sets, I/O bursts, and storage latency.
2. Identify hidden performance risks in average-only metrics.
3. Evaluate instance rightsizing against tail-latency SLOs.

Database Rightsizing Decision Framework

Utilization View (Can Be Misleading)



Actual Demand (What to Check)



Evaluate Before Downsizing

- Peak CPU, memory, and I/O usage
- Database working set (buffer cache)
- Storage latency and IOPS bursts
- p95 / p99 query latency vs. SLO
- Growth forecast and traffic seasonality

Risks of Premature Downsizing

- Higher tail latency (p95/p99)
- Increased I/O wait and query time
- Cache misses and memory pressure
- SLO violations during traffic spikes
- Cost savings offset by performance risk

Senior Judgment

Downsize only when the workload meets performance SLOs at both average and peak demand with acceptable tail latency.

Decision 43

Should engineering teams automatically delete unused AWS resources?

Key Points

1. Distinguish abandoned resources from standby, recovery, and compliance resources.
2. Examine resource ownership, dependencies, and retention requirements.
3. Design quarantine and approval processes before destructive cleanup.
4. Balance cost savings with recovery and operational requirements.

Safe Resource Cleanup Process

Discover Resources → Classify (Unused / Standby / Critical) → Quarantine (Tag & Isolate for Review) → Approve (Owner / Team) → Delete (Safe Cleanup)

Examples of Resources to Keep

- Disaster recovery and failover resources
- Standby environments (e.g., blue/green)
- Compliance or audit retention resources
- Low-usage but critical monitoring/logging
- Resources tied to scheduled or seasonal use

Examples That Can Be Removed (After Approval)

- Unatached EBS volumes and old snapshots
- Unused Elastic IPs
- Idle load balancers or target groups
- Stopped test instances (past retention period)
- Orphaned security groups and IAM roles

Architecture Trade-Off

Automated cleanup can reduce cost, but it must include classification, quarantine, and approval to avoid accidental loss of recovery or compliance capabilities.

Key Takeaway

Optimize costs with evidence, not assumptions: validate peak demand and business requirements before downsizing or deleting any resource.

Sustainability Is an Engineering Constraint, Not a Marketing Metric

Decision 44

Should noncritical workloads move to a different AWS Region for sustainability reasons?

Key Points

1. Evaluate regional carbon impact alongside latency, data residency, and availability.
2. Examine the effect of cross-region transfers and duplicated infrastructure.
3. Compare regional placement with workload scheduling improvements.
4. Balance sustainability goals with measurable operational constraints.

Regional Placement Trade-Offs

Region A (Lower Carbon)

- ✓ Lower carbon intensity (e.g., cleaner energy mix)
- ✓ Potential cost benefits
- ✓ Meets sustainability goals
- ✗ May increase user latency
- ✗ May have data residency or compliance limits

VS

Region B (Closer to Users)

- ✓ Lower latency and better user experience
- ✓ Fewer cross-region transfer costs
- ✓ Simpler operations
- ✗ Higher carbon intensity (depending on region)
- ✗ May not meet certain sustainability targets

Senior Judgment

Use a sustainability approach that also meets latency, compliance, availability, and cost requirements.

Evaluation Factors

Regional carbon intensity | Latency (SLOs) | Data residency | Cross-region transfer cost | Infrastructure duplication | Workload scheduling

Decision 45

Should applications retain detailed logs indefinitely for operational visibility?

Key Points

1. Examine logging volume, storage lifecycle, and query frequency.
2. Compare high-cardinality logs with actionable diagnostic value.
3. Identify privacy, security, cost, and resource-consumption consequences.
4. Define a retention strategy that meets operational and compliance needs.

Log Retention Decision Framework

Application Logs → Centralized Logging (CloudWatch / OpenSearch / S3) → Retention Policy (e.g., 7, 30, 90, 365 days) → Lifecycle (Archive / Delete (e.g., S3 Glacier))

Benefits of a Defined Retention Policy

- ✓ Faster and cheaper queries
- ✓ Meets compliance and audit needs
- ✓ Reduces storage and processing costs
- ✓ Limits exposure of sensitive data
- ✓ Improves overall system performance

Risks of Indefinite Log Retention

- ✗ Rapidly increasing storage costs
- ✗ Higher security and privacy risk (e.g., PII or sensitive data)
- ✗ Noisy, low-value data that is rarely used
- ✗ Longer search and analysis times
- ✗ Unnecessary resource consumption

Senior Judgment

Retain the evidence needed for operations, security, and compliance, not unlimited telemetry by default.

Key Takeaway

Be intentional about where workloads run and how long logs are kept—optimize for sustainability, reliability, security, and cost together.

The Review That Exposed Hidden Architectural Risk

Decision 46

Should a workload pass review when all infrastructure checks are green but recovery objectives remain unverified?

Key Points

1. Compare automated compliance results with demonstrated operational capability.
2. Examine failover exercises, dependency mapping, and restoration evidence.
3. Distinguish configuration compliance from actual resilience.

Compliance vs. Resilience

Infrastructure Checks (Automated)		Operational Resilience (Demonstrated)
✓ Configurations are compliant	VS	✓ Failover tested successfully
✓ Policies and standards met		✓ Recovery time (RTO) verified
✓ Resources correctly provisioned		✓ Data restoration (RPO) verified
✗ Does not prove recovery will work		✓ Dependencies and runbooks validated

Senior Decision

Record unverified recovery assumptions as architectural risks.

Decision 47

Should teams remediate every Well-Architected finding immediately?

Key Points

1. Prioritize findings by business impact, likelihood, exploitability, and recovery consequences.
2. Compare immediate remediation with controlled risk acceptance.
3. Evaluate remediation dependencies and the risks introduced by changes.

Finding Prioritization Framework

Factor	Example Consideration
Business Impact	Customer, revenue, compliance
Likelihood	How likely it can be exploited or cause failure
Exploitability	Ease and speed of exploitation
Recovery Consequence	Outage duration, data loss, service disruption

Remediation Options

- ✓ Immediate fix (critical / high risk)
- ✓ Planned remediation (with dependencies)
- ✓ Risk acceptance (with justification and review)

Senior Judgment

Risk reduction requires sequencing, ownership, and verification.

Decision 48

Should a high-risk architecture be accepted when replacing it would introduce greater short-term risk?

Key Points

1. Examine migration blast radius, technical debt, and operational maturity.
2. Define compensating controls, risk ownership, and time-bound acceptance.
3. Compare phased migration with immediate replacement.

Risk Management Decision

Replace Now (Higher Short-Term Risk)		Accept with Controls (Mitigated Risk)
✗ Larger change and blast radius	VS	✓ Keep the current architecture with compensating controls
✗ Higher chance of new issues		✓ Assign a clear risk owner
✗ Requires more testing and coordination		✓ Define a time-bound plan (e.g., 3–6 months)
✗ May disrupt business operations		✓ Migrate in phases to reduce risk

Senior Decision

A defensible exception requires evidence and an accountable owner.

Key Takeaway

A Well-Architected review is not just a compliance check — it is a risk-informed decision process that balances resilience, change risk, and business impact.

Final Architecture Review:
Defend 50 Decisions Under Production Constraints

Decision 49

Should a critical payments platform migrate from single-region Multi-AZ to multi-region active-active?

Production Scenario

A payments platform operates on AWS using EKS, RDS PostgreSQL, ElastiCache, SQS, and an Application Load Balancer. It experiences regional dependency failures, unpredictable traffic spikes, and occasional database connection exhaustion.

Architecture Context



Multi-AZ Already Addresses

- Instance and AZ failures (automatic failover)
- Higher availability within a region
- Managed database failover (RDS)
- Reduced impact from AZ-level outages

What Multi-AZ Does Not Address

- A full regional outage
- Regional service dependency failures
- Cross-region disaster recovery
- Traffic surges that exceed regional capacity

Key Decision Factors

- What Multi-AZ already protects.
- Replication, write ownership, and consistency.
- Recovery objectives (RTO/RPO).
- Operational complexity and team readiness.
- Cost impact and alternatives (active-active, warm standby, or stronger single-region).

Decision & Acceptance Criteria

Decision: Do not move to full active-active now.

Implement a multi-region warm standby (DR) with stronger single-region resilience.

Acceptance Criteria:

- RTO ≤ 15 minutes | RPO ≤ 5 minutes
- Handle ≥ 3× baseline traffic (load tested)
- Database connection pool and queue scaling in place
- DR failover tested and documented quarterly

Residual Risks

- Higher complexity than single-region
- Residual downtime during regional failover
- Ongoing DR cost and operational overhead

Decision 50

Should the architecture receive production approval with unresolved high-risk findings?

Review Scope

1. Evaluate all six AWS Well-Architected pillars.
2. Identify the five most consequential architectural risks.
3. Separate verified weaknesses from untested assumptions.
4. Rankremediation by blast radius, business impact, and implementation risk.
5. Define ownership, deadlines, and measurable verification.

AWS Well-Architected Pillars (Review Summary)

Operational Excellence | Security | Reliability | Performance Efficiency | Cost Optimization | Sustainability

Top 5 Architectural Risks (Example)

#	Risk	Impact	Priority
1	Regional outage (no active DR)	High	P1
2	Database connection exhaustion	High	P1
3	Insufficient autoscaling for traffic spikes	Medium	P2
4	Incomplete failure testing (recovery unverified)	Medium	P2
5	Excessive log retention (cost and privacy risk)	Medium	P3

Remediation Plan & Ownership

Risk	Owner	Deadline	Verification
#1	Platform Eng.	30 days	DR failover test
#2	Backend Eng.	14 days	Load & conn. test
#3	SRE	21 days	Autoscaling test
#4	SRE	30 days	Game-day exercise
#5	Platform Eng.	45 days	Log policy & cost report

Architecture Decision Record (ADR)

- **Chosen Design:** Single-region (Multi-AZ) with multi-region warm standby (DR).
- **Rejected Alternatives:** Full active-active (higher complexity and cost).
- **Accepted Risks:** Limited downtime during regional failover (with mitigation and timeline).